BotGraph: Graph Neural Network Analysis for Scalable Fake Profile and Botnet Detection

BIJJA NAGA LAKSHMI
Roll No. 23K91A0538
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

DIVVELA SRIVATSHAV
Roll No. 23K91A0564
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

ADUMULLA DURGA
Roll No. 23K91A0504
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

BURUGULA KOUSHIK
Roll No. 23K91A0550
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

NEELIMA
[DESIGNATION]
(project guide)
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

***Abstract*---**Fake profiles are created faster than moderators can remove them, and classifiers that look at one account at a time miss botnets whose members each look ordinary. This paper presents BotGraph, a system that treats accounts as nodes and interactions as edges, learns from each account's neighbourhood with GCN, GAT and GraphSAGE, finds coordinated groups by Louvain community detection over the learned embeddings, and lets moderators confirm or dismiss what is flagged. On a graph of 4,533 accounts and 94,456 ties, built from public Facebook, Instagram and Twitter data plus injected botnets, GraphSAGE reaches an F1 of 0.939 on 908 held-out accounts against 0.827 for the same features without the graph, while GCN adds only 1.0 point. All 8 injected botnets are recovered as single communities, and a training epoch on 36,264 accounts takes 0.39 s on a CPU. A feature ablation shows that the injected structure is easy, so the scores do not transfer to real platforms.

***Index Terms*---**botnet detection, fake profile detection, graph neural networks, GraphSAGE, Louvain communities, scalability, social network analysis.

I. INTRODUCTION

Social platforms carry a growing population of fake profiles and automated accounts that spread misinformation, inflate engagement and erode trust. Moderation by hand cannot keep pace, and per-account classifiers that read a profile or a posting history in isolation have two blind spots. They miss coordinated networks whose members are each made to look legitimate, and they go stale as bot behaviour changes. A platform is naturally a graph, and a coordinated network leaves a trace in that graph even when each profile looks normal.

Today, large platforms combine user reports, rule-based filters and per-account classifiers. The cost is twofold. Rules and signatures are fixed by what moderators have already seen, so a new bot behaviour passes until someone writes the next rule. Per-account scores, even good ones, give a reviewer a flat list of suspects with no sign of which belong together, so a network of fifty accounts is reviewed fifty times instead of once. The target users here are the trust-and-safety teams of large regional platforms, who need both a ranked account list and a ranked list of groups, with a reason attached to each score.

Graph neural networks (GNNs) learn from an account's neighbourhood as well as from its own attributes, and have become the mainstream approach to bot detection [2], [3], [4], [5]. The base paper of this work, PruferNET [1], augments GraphSAGE node features with Prufer-sequence statistics of a minimum spanning tree of the Facebook graph, but it labels nodes by their depth in that tree: it measures influence, not fakeness, and produces neither bot labels nor groups. The gap this project addresses is to apply the same relational learning to fake-profile and botnet detection under heavy class imbalance, to expose coordinated clusters, to close the loop with human moderators, and to measure how the cost grows with the graph.

The contributions are: (1) a graph builder that joins real Facebook topology with labelled profile and posting rows and injected botnets, with 23 node features in four groups; (2) a like-for-like comparison of GCN, GAT and GraphSAGE against a no-graph baseline on one split with three seeds; (3) botnet cluster detection from the learned embeddings, scored against a known answer, with a moderator queue whose decisions feed retraining without touching the test split; and (4) a scalability sweep and an honest ablation of what the benchmark rewards. Section II reviews related work, Section III describes the method and the implementation, Section IV reports the results, and Section V concludes.

A. System Hardware

All training, evaluation and timing ran on one machine: a 12th Gen Intel Core i5-12500 (6 cores, 12 threads), 16 GB RAM, Windows 11 Pro, CPU only. No GPU was used. The machine was shared with other jobs while the measurements were taken, so all times are upper bounds.

B. Software Requirements

Python 3.11 with PyTorch (CPU build) [6], NumPy, pandas and scikit-learn for metrics, and NetworkX [7] for Louvain communities. The service uses FastAPI, SQLAlchemy 2 over SQLite, PyJWT and bcrypt; the web app uses React 18 and Vite 5. The three graph layers are written directly against sparse PyTorch tensors, because torch-geometric wheels frequently fail to build on Windows. The whole system runs offline on Windows with Python and Node.js installed.

II. LITERATURE SURVEY

The base paper [1] (IEEE Access, 2026) builds a minimum spanning tree of the SNAP Facebook graph (4,039 nodes, 88,234 edges), computes its Prufer sequence, and derives label frequency, entropy, unique-label ratio, transition influence and subtree depth. These are added to degree, closeness, PageRank and a neighbour-PageRank trust score, and a GraphSAGE model (64 hidden units, dropout 0.5, AdamW at 0.003, 300 epochs, 60/20/20 split) assigns each node to one of three depth classes. It reports 99.63 % test accuracy with the Prufer features against 96.54 % without them, and repeats the check on Email-Eu-core. Its limits for this project are that the label is derived from the graph itself, there are no fake or bot labels, no clusters, no moderator loop and no scalability measurement. Its useful lessons are the SNAP Facebook data, GraphSAGE as a backbone, and the case that global structure adds information a per-node model lacks.

Bot detection with GNNs is the closest line of work. BotRGCN [8] applies relational graph convolution on Twitter. Spectral and multi-scale filters were proposed in [9] and, with a homophily-aware Gaussian window, in HW-GNN [3], which reports a 4.3 % average F1 gain over existing methods on Cresci-15, TwiBot-20 and MGTAB. RABot [4] attacks the two problems this project also faces, class imbalance and topological noise, with neighbourhood-aware oversampling and reinforcement-learned edge filtering. Temporal models add time: BotDGT [5] uses dynamic graph transformers and BotTGCL [10] adds graph contrastive learning. LGB [11] adds a language model for sparsely connected accounts and reports gains of up to 10.95 % over its baselines. These works score individual accounts on public benchmarks; none reports coordinated-group recovery, a human review loop, or timing as the graph grows. The backbones compared here are the standard GCN [12], GAT [13] and GraphSAGE [14].

TABLE I
REPRESENTATIVE RELATED WORK

Work	Approach	Data	Reported result	Limitation
PruferNET [1]	MST Prufer features + GraphSAGE	SNAP Facebook; Email-Eu-core	99.63 % vs 96.54 % acc. (depth classes)	Influence label, no bots
BotRGCN [8]	Relational GCN	Twitter bot data	85.5 F1 on TwiBot-20 (as tabulated in [4])	Per account; no groups
HW-GNN [3]	Gaussian-window spectral GNN	Cresci-15, TwiBot-20, MGTAB	+4.3 % mean F1 over prior methods	No groups or review loop
RABot [4]	Oversampling + RL edge filter	Same three sets	98.07 F1 (GCN) on Cresci-15	No groups; extra RL stage
BotDGT [5]	Dynamic graph transformer	Cresci-15, TwiBot-20	98.3 / 87.8 F1 (as tabulated in [4])	Needs time snapshots
LGB [11]	Language model + GNN	Two real datasets	Up to 10.95 % over baselines	Needs account text
BotGraph	GCN, GAT, GraphSAGE + Louvain	Facebook ties + injected botnets	F1 0.939 (this paper)	Partly synthetic data

Two points from this literature shape the design here. First, most benchmarks are text- and metadata-rich, so a method that reads only graph structure and cheap profile features offers a cheaper operating point for platforms that cannot afford a language model per account. Second, imbalance is handled in the literature by changing the data (oversampling, as in [4]); this project changes the loss instead and keeps the split fixed, which makes the comparison between architectures easy to audit. None of the cited works evaluates what a moderator does with a flagged account, which is why the review queue is part of the system and not an afterthought.

The research gap is therefore the combination: relational learning for fake accounts under imbalance, cluster recovery measured against a known answer, a moderator loop whose labels never leak into the test split, and an explicit scalability measurement, all reproducible on one CPU.

III. METHODOLOGY

A. Graph and Data

Accounts are nodes and ties are undirected edges. The topology is the SNAP Facebook ego-network collection [15]: 3,963 accounts and 88,156 ties. Each account receives a profile row resampled from a labelled Instagram fake-account set [16] and a posting row from a Twitter bot set [17]. The two sources describe different people, so they are combined by resampling, which keeps realistic value ranges but not real correlations. To measure cluster recovery against a known answer, 8 botnets of 34-62 accounts and 150 lone spammers are injected (570 accounts, 6,300 ties): botnet members are tied pairwise with probability 0.28 and follow 2-7 real accounts each, and lone spammers follow 3-10. Of the injected accounts, 198 are given a genuine-looking profile and 231 genuine accounts a fake-looking one, so that the profile alone cannot solve the task. The graph has 4,533 nodes, 94,456 ties and 570 fake accounts (12.6 %), split 60/20/20 by label into 2,719 / 906 / 908 accounts, identical for every model.

The injection is a design choice, not a claim about real botnets. Real labelled botnets with a public graph are rare, and without a known answer cluster recovery cannot be scored at all. Camouflage is added because otherwise the profile alone would solve the task and the graph would add nothing. The injected structure still makes the task easier than a real platform, as the ablation in Section IV shows.

B. Node Features

Each account has 23 standardised features: 12 profile features (picture, digits in the handle, bio length, log posts, followers and following and their ratio, among others), 4 posting features, 6 temporal features (account age, posts per day, burstiness, peak-hour share and its sine and cosine) and 1 interaction frequency. Botnet members share a creation window and a peak posting hour, so those traits are common to a cluster without making any member stand out. The temporal and interaction features and the handles are synthetic and seeded.

C. Models and Training

Two message-passing layers of 64 units are followed by a linear head. With $\hat{A}$ the symmetrically normalised adjacency with self-loops, the layers are

$$GCN:\ H' = ELU(\hat{A}\, H\, W) \qquad (1)$$

$$SAGE:\ h'_v = ELU(\ W_s\, h_v + W_n\, mean(\ h_u : u \in N(v)\ )\ ) \quad (2)$$

GAT scores each tie and normalises the scores over the neighbourhood of the receiving node,

$$alpha_{vu} = softmax_u(\ LeakyReLU(\ a^{\wedge}T\ [\ W\, h_v \,||\, W\, h_u\ ]\ )\ ) \quad (3)$$

It uses four attention heads and then one head, with the edge softmax computed by scatter operations. The feature-only baseline replaces the layers with dense ones, so any difference comes from the graph. All models use dropout 0.3, Adam [18] (learning rate 0.01, weight decay 5e-4), a class-weighted cross-entropy whose positive weight is the square root of the class ratio, up to 160 epochs and early stopping on validation F1. The decision threshold is tuned on the validation set. Every model is trained with three seeds and the test split is scored once.

Because only one account in eight is fake, the loss is weighted rather than the data resampled. With n_neg genuine and n_pos fake training accounts, the weights are

$$w_0 = 1, \quad w_1 = sqrt(\ n_{neg} / n_{pos}\ ) \quad (4)$$

which raises the cost of a missed fake account without letting the minority class dominate. A deliberately small configuration keeps the whole pipeline within minutes on a laptop CPU.

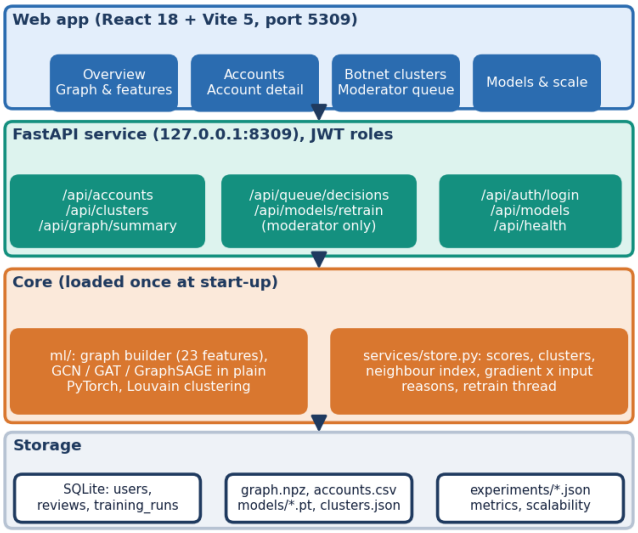


Fig. 1. System architecture: web app, FastAPI service, core modules loaded once at start-up, and storage.

D. Botnet Clusters and Moderator Loop

Accounts scored at 0.30 or more are candidates. Each is linked to its five most cosine-similar candidates in embedding space, and every real tie between two candidates adds weight 1.0. Louvain communities [19] are found on this weighted graph and groups under five accounts are dropped. A group whose tie density is at least 5 % is called coordinated and otherwise loose. Moderators confirm or dismiss an account or a whole cluster. Each decision is stored with the model score at that time, becomes a training label at the next run, and moves the reviewed accounts into the training set; accounts of the test split are never touched, so F1 before and after retraining is measured on the same accounts. Reasons for a score use gradient times input on the fake-versus-genuine margin, a first-order approximation.

E. Implementation

The artefacts (graph, scores, embeddings, clusters, metrics) are loaded once when the FastAPI service starts. Eighteen endpoints serve the web app; sign-in returns a 12-hour JWT, passwords are bcrypt hashes, and analysts receive HTTP 403 on decision and retraining routes. SQLite holds three tables: users, reviews and training runs. Retraining runs on a background thread, one at a time.

The web app has ten screens: landing, sign-in, overview, graph and features, ranked accounts, account detail with reasons and a neighbourhood graph, cluster list, cluster detail, the moderator queue with a decision log, and a model comparison with a scalability view. Every screen has loading, empty and error states. The smoke test starts its own API on a spare port with a scratch database and scratch copies of the model files, so the shipped artefacts are never altered.

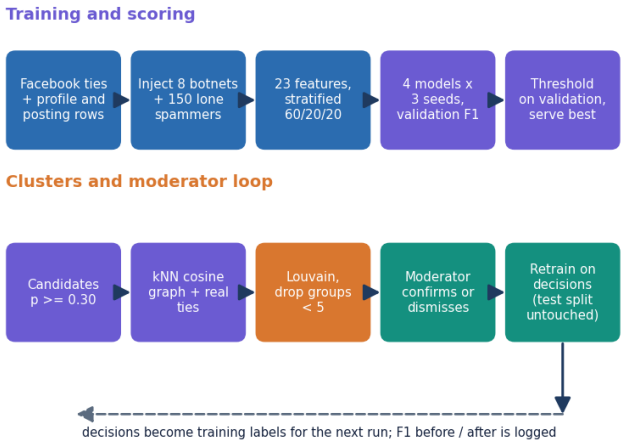


Fig. 2. Main flow: training and scoring (top), cluster detection and the moderator loop (bottom).

F. Evaluation Protocol

Precision P = TP/(TP+FP), recall R = TP/(TP+FN) and F1 = 2PR/(P+R) are computed for the fake class on the held-out split, together with accuracy, ROC-AUC and PR-AUC. Cluster quality is scored by matching each injected botnet to the detected community that overlaps it most and reporting that community's recall and precision. Scalability is measured by copying the account graph one, two, four and eight times as disjoint components and timing ten training epochs and one forward pass per model. A threshold sweep and a feature-group ablation (three seeds each) complete the evaluation. The ablation retrains models in memory and leaves the served models untouched.

IV. RESULTS

Conditions: the 908-account test split holds 114 fake accounts (12.6 %), so a model that answers genuine for everyone scores 87.4 % accuracy and F1 is the headline metric. Table II gives means and standard deviations over

three seeds, with thresholds tuned on validation data. The graph models were trained on a CPU in 3-41 s per model on average.

TABLE II
TEST-SPLIT RESULTS, MEAN OF 3 SEEDS (FP / FN: SEED 0, 908 ACCOUNTS)

Model	Acc.	Prec.	Rec.	F1	AUC	FP / FN
MLP baseline	0.959	0.868	0.789	**0.827 ± 0.010**	0.967	12 / 24
GCN	0.964	0.958	0.743	**0.837 ± 0.009**	0.988	3 / 28
GAT	0.977	0.963	0.845	**0.900 ± 0.018**	0.978	3 / 18
GraphSAGE	0.985	0.975	0.906	**0.939 ± 0.006**	0.990	3 / 9

All three graph models beat the baseline on F1, but unevenly. GraphSAGE gains 11.2 points (0.975 precision, 0.906 recall, 3 false positives and 9 false negatives for seed 0). GAT gains 7.3 points with the widest seed spread (F1 0.877 to 0.922). GCN gains 1.0 point, and its recall (0.743) is below the baseline's (0.789); its gain comes from precision. A plausible reason, not tested here, is that GraphSAGE keeps the account's own features in a separate weight from the neighbourhood mean, whereas GCN's normalised smoothing mixes an account into its hubs. The result does not hinge on the tuned threshold: GraphSAGE F1 stays between 0.927 and 0.952 for every threshold from 0.30 to 0.60.

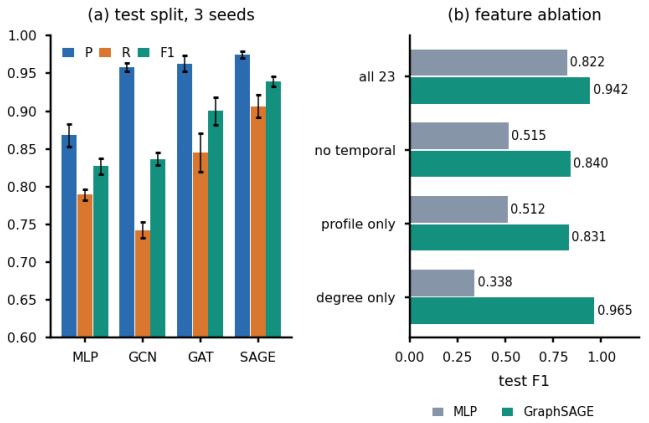


Fig. 3. (a) Precision, recall and F1 of the four models. (b) Test F1 when feature groups are removed or used alone.

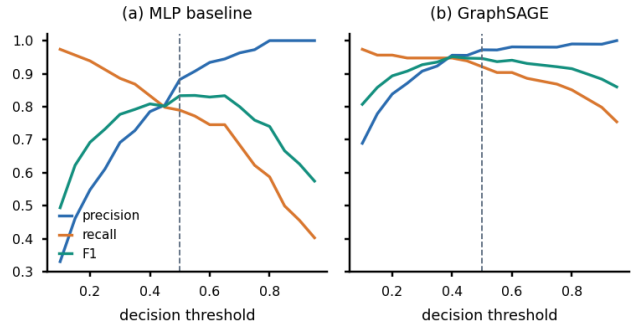


Fig. 4. Precision, recall and F1 against the decision threshold on the test split (dashed line: served threshold 0.5).

The sweep makes the value of the graph concrete. To reach a recall of 0.90 the MLP baseline must accept a precision of about 0.61 (threshold 0.25, 66 false positives among 170 flagged accounts), whereas GraphSAGE keeps a precision of 0.87 at a recall of 0.95 (threshold 0.25, 16 false positives among 124 flagged). The baseline's best F1 on the test split is 0.834; the F1 curve of GraphSAGE is flat across a wide band, so the operating point can be moved towards recall or precision without much loss.

For a moderator the sweep translates into workload. On the test split, the served threshold of 0.5 flags 108 accounts for review, of which 105 are fake; lowering the threshold to 0.30 flags 119 accounts, of which 108 are fake, so eleven more accounts are reviewed to find three more fakes. Because a cluster decision applies to every member at once, the review cost of a coordinated group is one decision rather than one per account, which is the practical benefit of exposing clusters in addition to scores. This workload estimate comes from seed 0 on 908 accounts and is indicative only.

Botnets. Louvain over the embeddings finds 9 groups. All 8 injected botnets are recovered as single communities (mean recall 1.00, mean precision 0.988, lowest precision 0.923), plus one loose group of 160 lone spammers and look-alikes with a tie density of 0.1 %. This is a known-answer test on seeded botnets with dense internal ties, so it shows that the method finds this kind of coordination, not that it finds real botnets.

TABLE III
RECOVERY OF EACH INJECTED BOTNET BY ITS BEST-MATCHING COMMUNITY

Botnet	Size	Recall	Precision
1	62	1.000	1.000
2	50	1.000	1.000
3	52	1.000	1.000
4	60	1.000	0.923
5	48	1.000	1.000
6	56	1.000	1.000
7	58	1.000	0.983
8	34	1.000	1.000

Scalability. Copying the graph as disjoint components grows it from 4,533 accounts and 94,456 ties to 36,264 accounts and 755,648 ties. A GraphSAGE epoch grows from 0.060 s to 0.392 s and inference from 0.027 s to 0.186 s, roughly in proportion to the ties; a GAT epoch grows from 0.255 s to 2.022 s. The largest graph is still far below platform scale, and disjoint copies understate the cost of a well-mixed graph. The API answers its ten read endpoints with median latencies of 3-16 ms (95th percentile at most 28 ms), and the 36-check smoke test passes.

TABLE IV
SECONDS PER TRAINING EPOCH / PER FORWARD PASS ON CPU

Accounts / ties	GCN	GAT	GraphSAGE
4,533 / 94,456	0.142 / 0.036	0.255 / 0.051	0.060 / 0.027
9,066 / 188,912	0.152 / 0.065	0.597 / 0.129	0.115 / 0.054
18,132 / 377,824	0.307 / 0.143	1.103 / 0.259	0.203 / 0.099
36,264 / 755,648	0.577 / 0.235	2.022 / 0.584	0.392 / 0.186

The sweep shows GraphSAGE as the cheapest and most accurate graph model here, and GAT as the most expensive: at the largest size a GAT epoch costs about 5.2 times a GraphSAGE epoch. Eight times the graph costs GraphSAGE about 6.5 times the time per epoch, which supports an

approximately linear trend in the number of ties over this range only.

Testing. The 36 smoke-test checks drive the real API end to end: sign-in and roles (anonymous requests and a wrong password receive 401, analysts receive 403 on decisions and retraining), graph counts and feature summaries, the three models beating the baseline, ranking, reasons and neighbourhood graphs, cluster recovery, queue rules (a cluster decision clears its members from the queue, a decision can be undone only until a training run has used it) and the retraining bookkeeping (F1 before and after recorded for every model, the retrained model served, the full graph still served). There is no separate unit-test suite, no load test and no test against adversarial accounts.

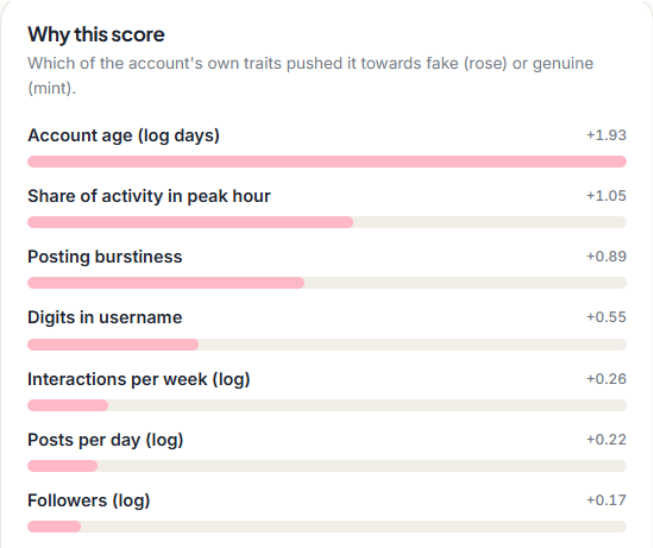


Fig. 5. Output screen: the reasons behind one account's fake probability, as bars of gradient-times-input contribution.

Cluster	Type	Accounts ↓	Avg fake probability	Tie density
BN-00055	Loose group	160	75%	0%
BN-00112	Coordinated	65	96%	26%
BN-03963	Coordinated	62	99%	31%
BN-03936	Coordinated	59	100%	30%
BN-04235	Coordinated	56	100%	29%
BN-04075	Coordinated	52	100%	29%
BN-04025	Coordinated	50	100%	31%
BN-04187	Coordinated	48	100%	33%

Fig. 6. Output screen: detected groups with size, mean fake probability and tie density; BN-00055 is the loose group.

Reading the output screens. Fig. 5 shows the first reasons for a flagged account whose own traits are weak but plainly unusual: a short account life, a posting hour that carries a large share of its activity, bursty posting and digits in the handle each push the fake margin up, and the app adds a separate figure for how much the account's neighbours push it. Fig. 6 lists the groups: eight coordinated clusters with tie densities of roughly 26-33 %, creation windows of 11-17 days and a shared peak hour that holds 94-100 % of members, next to the loose group whose members share almost no ties. A moderator can open a group, see its graph and members, and confirm or dismiss all of it with one decision.

Threats to validity. The data are partly synthetic and the labels of injected accounts are exact by construction, so label noise, which is the second problem named in the problem statement, is not tested. The split, the seeds and the thresholds are few: three seeds and one split give a standard deviation but no confidence interval, and the confusion matrices shown are for one seed. GAT's edge softmax sums in a non-deterministic order, so its figures can move by about 0.01 between runs. The reasons shown to moderators are gradient-times-input attributions, which are an approximation, not a proof of cause.

Comparison with the base paper. PruferNET [1] reports 99.63 % and 96.54 % accuracy for a three-class depth labelling of a Facebook graph. BotGraph reports 0.985 accuracy and 0.939 F1 for a binary fake-account label on 4,533 accounts that are partly synthetic. The target, the labels and the data differ, so the numbers are not comparable and neither work outranks the other. What carries over is the data family, GraphSAGE as backbone and the 60/20/20 split.

Failure cases. The served model caught all 82 botnet members in the test split (the baseline 64) but only 23 of 32 lone spammers (the baseline 26). All 9 missed fake accounts are lone spammers with 4-8 ties; the three false positives are genuine accounts. The graph therefore helps against coordination and not against isolated accounts.

The ablation is the main caution. The synthetic temporal group is the most valuable attribute group: without it the baseline falls from 0.822 to 0.515 and GraphSAGE from 0.942 to 0.840, while posting behaviour adds nothing. GraphSAGE on a single feature, the log degree, reaches F1 0.965, above the 0.942 of the full model, while the same feature alone gives the MLP 0.338. The injected botnets and spammers have a distinctive local degree pattern, so neighbourhood information nearly solves this benchmark. The ablation retrains in memory, so its all-feature row differs from Table II in the third decimal.

Objectives against outcomes. Of the six objectives in the project brief, the graph representation, node features, three-way comparison, cluster detection, evaluation against labels and scalability sweep are all met on this benchmark. Two qualifications apply: the feature objective is met without text or linguistic features, although the project abstract mentions behavioural language cues, and the cluster objective is met only for injected botnets. No objective was dropped, and the headline numbers are reported with the ablation that limits how far they can be trusted.

V. CONCLUSION

BotGraph meets its six objectives on its own benchmark: a graph of 4,533 accounts, 23 features, a three-way

comparison with a no-graph baseline, botnet clusters from embeddings, all six metrics, and a scalability sweep. GraphSAGE is the best model (F1 0.939 against 0.827), GAT is second and costs three to five times as much, and GCN barely helps. All 8 injected botnets were recovered. The honest limits are large: the botnets are injected, the temporal features are synthetic, profile and posting rows come from different people, there are no text features, and the degree-only result shows that the benchmark rewards structure more than a real platform would. Scalability was measured only to 36 thousand accounts, and no formal user test or adversarial test was run. Future work should add MST and Prufer-style structural features from the base paper, text features, temporal graphs, neighbour sampling and GPU training for platform scale, harder botnets with less distinctive degree patterns, and labelled real botnets. The moderator loop is implemented and bookkeeping-tested, but its effect on accuracy over many review rounds was not studied. A pytest suite, cross-validation and tests against evasive bots that copy normal degree patterns would be the next steps in evaluation.

REFERENCES

[1] A. Saha, N. K. Sarkar and S. Pratihar, "PruferNET: An augmented graph neural network for influential node detection in social networks," *IEEE Access*, vol. 14, pp. 79940-79959, 2026, doi: 10.1109/ACCESS.2026.3695637.

[2] S. Cresci, "A decade of social bot detection," *Commun. ACM*, vol. 63, no. 10, pp. 72-83, Oct. 2020.

[3] Z. Liu, J. Gao, Z. Ji and L. Zhao, "HW-GNN: Homophily-aware Gaussian-window constrained graph spectral network for social network bot detection," arXiv:2511.22493, Nov. 2025.

[4] L. Zhang, X. Wang, H. Du, Y. Xu, Z. Liu and Y. Liu, "RABot: Reinforcement-guided graph augmentation for imbalanced and noisy social bot detection," arXiv:2602.21749, Feb. 2026.

[5] B. He, Y. Yang, Q. Wu, H. Liu, R. Yang, H. Peng, X. Wang, Y. Liao and P. Zhou, "BotDGT: Dynamicity-aware social bot detection with dynamic graph transformers," arXiv:2404.15070, Apr. 2024 (IJCAI 2024 per the arXiv record).

[6] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, arXiv:1912.01703.

[7] A. A. Hagberg, D. A. Schult and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python in Science Conf. (SciPy)*, 2008, pp. 11-15.

[8] S. Feng, H. Wan, N. Wang and M. Luo, "BotRGCN: Twitter bot detection with relational graph convolutional networks," in *Proc. IEEE/ACM Int. Conf. Adv. Soc. Netw. Anal. Min. (ASONAM)*, 2021, pp. 236-239.

[9] S. Shi, K. Qiao, Z. Wang, J. Yang, B. Song, J. Chen and B. Yan, "Multi-scale graph neural network with signed-attention for social bot detection: A frequency perspective," arXiv:2307.01968, Jul. 2023.

[10] W. Wang, T. Zang and X. Zhang, "Temporal-aware social bot detection with graph contrastive learning," in *Computational Science - ICCS 2025*, Springer, 2025.

[11] M. Zhou, D. Zhang, Y. Wang, Y. Geng, Y. Dong and J. Tang, "LGB: Language model and graph neural network-driven social bot detection," arXiv:2406.08762, Jun. 2024.

[12] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017, arXiv:1609.02907.

[13] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018, arXiv:1710.10903.

[14] W. L. Hamilton, R. Ying and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 1024-1034.

[15] J. McAuley and J. Leskovec, "Learning to discover social circles in ego networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012, pp. 539-547.

[16] free4ever1, "Instagram fake spammer genuine accounts," Kaggle dataset, licence CC BY 3.0, https://www.kaggle.com/datasets/free4ever1/instagram-fake-spammer-genuine-accounts (accessed Oct. 2026).

[17] goyaladi, "Twitter bot detection dataset," Kaggle dataset, licence CC0, https://www.kaggle.com/datasets/goyaladi/twitter-bot-detection-dataset (accessed Oct. 2026).

[18] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015, arXiv:1412.6980.

[19] V. D. Blondel, J.-L. Guillaume, R. Lambiotte and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Stat. Mech.: Theory Exp.*, no. 10, p. P10008, 2008.